AI 智能体学习路线：从零基础到独立完成项目

> 适合没有智能体经验、愿意学习 Python 的初学者。建议 12 周，每周投入 6～10 小时。
>
> 最终目标：完成一个能检索个人文档、调用工具、多步执行任务，并具备测试、安全和部署能力的学习助手。

最终项目：个人知识库学习助手

项目功能：上传 Markdown、PDF 或 TXT；针对资料提问；回答附来源；支持计算、日期查询或网页搜索工具；资料不足时承认不知道；记录请求、工具调用、错误和耗时；提供简单网页或 API。

路线总览

周次	学习内容	阶段产出
1	Python、命令行、Git、虚拟环境	文本统计程序与项目骨架
2	HTTP、JSON、模型 API	命令行问答助手
3	提示词、结构化输出、校验	邮件分类器与信息提取器
4	工具定义、参数校验、错误处理	可调用计算器和文件搜索的助手
5	任务拆解、工作流、状态、人工确认	会议纪要处理器
6	文档解析、分块、检索	本地文档检索程序
7	Embedding、向量检索、引用	RAG 知识库问答
8	多轮对话、会话状态、上下文管理	有会话状态的助手
9	测试集、评估、日志、安全	可重复评估的项目版本
10	FastAPI、接口设计、简单页面	可通过 API 使用的应用
11	配置、部署、监控、费用控制	部署版本
12	综合项目、文档、演示、复盘	可展示的完整作品

第 1 周：Python 与开发环境

学什么

- Python 变量、条件、循环、函数、列表、字典。
- 文件读写、JSON、异常处理、模块。
- 命令行、虚拟环境、pip、Git 的基本操作。

- 环境变量与密钥管理。

怎么做

1. 安装 Python 3.11+ 和代码编辑器。
2. 创建项目目录与虚拟环境。
3. 建立 `requirements.txt`、`.gitignore` 和 `.env.example`。
4. 写程序读取 `notes.txt`，统计行数、字符数和关键词频率，并把结果保存为 JSON。

产出与检查

项目能从命令行运行；依赖可安装；程序能处理文件不存在等错误；API 密钥不会写入代码或提交到 Git。

第 2 周：模型 API 与对话

学什么

- HTTP 请求与响应、JSON、API Key、状态码。
- 大语言模型、Token、上下文窗口、系统消息与用户消息。
- 模型输出可能出错的原因；请求超时和限流。

怎么做

1. 阅读所选模型服务商的官方 API 文档。
2. 从 `.env` 读取 API Key。
3. 写命令行程序，接收用户输入、调用模型并打印回答。
4. 捕获网络/API 错误，记录耗时。
5. 保存多轮对话历史，提供清空和退出命令。

产出与检查

完成可连续对话的命令行助手。能够解释上下文来自请求消息，而不是模型自动永久记住；API 故障时程序有可读提示。

第 3 周：提示词与结构化输出

学什么

提示词应明确角色、目标、输入、限制、输出格式和信息不足时的处理规则。学习 JSON、JSON Schema 或 Pydantic 校验。

怎么做

制作客服邮件分类器，分类为售前、订单、退款、技术故障、投诉或其他，并返回类别、紧急程度、理由和置信度。再制作一个从文本抽取姓名、日期、金额和待办事项的信息提取器。

准备至少 20 条输入，包含正常、缺字段、歧义、格式混乱和无关内容。将模型输出解析并校验，不要只把字符串直接展示。

产出与检查

得到可解析的结构化数据；对缺字段、非法 JSON 和置信度越界有处理；有固定测试样例并记录失败原因。

第 4 周：工具调用

学什么

理解模型提出工具名称和参数、程序校验并执行、结果再交回模型的完整流程。学习函数说明、参数 Schema、权限、超时和错误处理。

怎么做

用普通 Python 函数实现：

- 计算器（限制可执行表达式，禁止不受限 `eval`）。
- 日期计算器。
- 本地文件搜索。
- 可选：天气或公开信息查询 API。

让模型根据请求选择工具。程序必须验证工具名称与参数，记录调用日志，并处理工具失败。

产出与检查

完成生活事务助手。能清楚区分“模型决策”和“程序执行”；发送、删除、付款等有副作用操作不得由模型未经确认直接执行。

第 5 周：工作流、状态与人工确认

学什么

复杂任务拆成可检查步骤；每步有明确输入和输出；失败能定位；学习任务状态、暂停与恢复、人工确认节点。

怎么做

实现会议纪要处理器：读取文本、提取主题和结论、提取待办、整理负责人和截止时间、输出 Markdown。生成待办后先展示给用户确认或修改，再生成最终清单。

不要把整个任务塞进一次模型调用。中间结果应可检查、可修改。

产出与检查

具备至少三步的工作流；步骤失败可定位；危险操作可暂停并等待人工批准。

第 6 周：RAG 基础与文档检索

学什么

RAG（检索增强生成）的两条流程：

```
建库：读取文档 -> 清洗 -> 分块 -> 向量化 -> 保存索引和元数据
问答：用户问题 -> 检索片段 -> 交给模型 -> 生成带来源回答
```

理解文档解析、分块、Embedding、相似度、元数据和检索数量。

怎么做

准备 5～10 篇 Markdown 或 TXT 文档，按段落或标题切分，保留文件名与标题。先只展示检索结果，不接入生成模型。人工检查结果是否相关。

产出与检查

能够针对问题找到相关片段；能区分检索错误与模型生成错误。

第 7 周：完成 RAG 问答

怎么做

1. 为片段生成 Embedding 并建立向量索引。
2. 根据用户问题检索相关内容。
3. 将片段交给模型，要求只根据片段回答。
4. 显示来源文件或标题。
5. 没有足够依据时明确回答“资料中没有找到”。
6. 测试答案在单一文档、需综合多个片段、文档外、同义表达和误召回问题。

每次只调整一个参数，例如分块大小、重叠量、检索数量或重排序方式，并记录测试结果。

产出与检查

回答能追溯到来源；文档外问题不会强行编造；能说明检索到哪些证据。

第 8 周：会话记忆与状态管理

学什么

区分当前上下文、当前会话状态和长期记忆。长期记忆只保存必要且获准的信息；不要无限保存聊天内容或敏感数据。

怎么做

为助手增加会话 ID、近期对话、最近检索结果、任务状态和清空会话功能。上下文太长时，截断不重要消息或先总结旧消息。

产出与检查

助手能理解连续追问；历史不会无限增加成本；用户可以清除会话数据。

第 9 周：评估、日志与安全

建立测试集

覆盖正常问题、边界问题、信息不足、文档外问题、提示词注入、越权请求、工具失败和超长输入。每条记录输入、期望行为、实际输出、通过与否、失败原因。

评估指标

按项目选择任务成功率、分类/提取准确率、检索命中率、引用正确率、工具调用正确率、拒答准确率、平均延迟和单次成本。不要只凭回答是否流畅来判断。

日志与安全

记录请求 ID、模型、耗时、工具名称、检索文档 ID、错误类型和估算成本；避免保存不必要的敏感内容。将网页和文档视为不可信输入；限制工具权限；设置超时、重试上限和费用上限；高风险操作要求人工确认。

产出与检查

有可重复运行的测试集，能定位失败步骤，并能说明应用权限、数据和费用方面的限制。

第 10 周：封装 API 与简单界面

学什么

HTTP 方法、JSON 请求响应、路由、参数校验和错误码。建议使用 FastAPI。

怎么做

实现接口：

```
POST /chat
POST /documents
GET  /documents
GET  /health
DELETE /sessions/{session_id}
```

页面至少提供输入框、回答区、来源展示、工具状态、错误提示和清空会话按钮。

产出与检查

命令行程序变成可通过 HTTP 使用的应用；接口参数经过校验；错误信息对用户可理解。

第 11 周：部署与运维基础

学什么

生产环境变量、密钥管理、日志、数据持久化、Docker 或托管服务、HTTPS、访问控制和费用限制。

怎么做

部署到可访问环境。检查密钥不在前端、文件上传有类型和大小限制、接口有限流和超时、调试信息不对外暴露、用户数据有清理策略。

产出与检查

其他人能访问应用；你能查看日志并定位错误；你能解释部署费用和数据风险。

第 12 周：综合项目与作品整理

项目要求

完成个人知识库学习助手，至少包含一个模型调用、一个知识库或两个工具、明确的输入输出、10 条以上自动化测试、日志、错误处理和人工确认机制。

整理作品

README 写清楚目标用户、架构、启动方式、配置方法、测试方式、安全限制、已知问题和费用估算。准备演示数据与简短演示流程。

项目复盘

回答：哪些步骤由代码完成、哪些交给模型？常见错误是什么？检索错误和生成错误如何区分？一次请求耗时和费用多少？下一版最值得改进什么？

每周学习节奏

- 约 20%：阅读一个主题的官方文档或教程。
- 约 40%：编写最小可运行代码。
- 约 30%：测试异常输入和失败案例。
- 约 10%：记录结果、整理 README 和下一步计划。

每周保留一个可运行版本。记录失败案例，不要只保存成功演示。

推荐起步技术栈

- Python；模型服务商官方 SDK。
- Pydantic 或 JSON Schema 做数据校验。
- FastAPI 提供 API。
- 先用简单本地索引学习 RAG，再按需求选择向量数据库。
- 先用普通 Python 函数和状态对象组织工作流，复杂后再选框架。
- Git 做版本管理。

框架和 SDK 会变化。长期更重要的是任务拆解、工具边界、检索、测试与安全。

初学阶段暂缓

多智能体协作、复杂自主规划、大规模模型训练、从零实现 Transformer、同时学习多个框架，以及没有测试就追求全自动执行。

最终自测

- [ ] 会创建虚拟环境、管理依赖和安全读取密钥。
- [ ] 会调用模型 API、管理多轮上下文并处理失败。
- [ ] 会让模型输出可校验的结构化数据。
- [ ] 会定义有参数校验和权限限制的工具。

- [ ] 会设计多步骤工作流、状态和人工确认节点。
- [ ] 会构建带来源引用、可拒答的 RAG 系统。
- [ ] 有固定测试集、日志、延迟和成本记录。
- [ ] 能说明提示词注入、越权和敏感数据风险。
- [ ] 能部署应用并说明其限制。

最简执行版

时间有限时依次完成：

1. 学 Python、JSON 和环境变量。
2. 调用一次模型 API。
3. 输出并校验 JSON。
4. 封装计算器或文件搜索工具。
5. 实现一个三步工作流。
6. 为几篇文档做 RAG 问答。
7. 为 20 个问题建立测试集。
8. 用 FastAPI 或简单页面部署。

总结：先稳定调用模型，再让模型调用工具；随后让它基于可靠资料工作，最后补上状态、评估、安全和部署。每一步都用可运行的小项目验证。